
Assignment 3 Report: CPU Scheduling

1 Introduction [2']

In this assignment, we are required to work on the `xv6-riscv` OS kernel and run it on the QEMU emulator, implementing a Multi-Level Feedback Queue (MLFQ) scheduler to replace the original single-queue Round Robin (RR) scheduler in `xv6`. In addition to the core scheduling part, I also implement **Bonus 1**: a real-time graphical monitor running on the host GUI that visualizes the states and queue levels of all `xv6` processes while the system is running.

The goal of the new scheduler is to better differentiate CPU-bound and I/O-bound processes and thereby improve overall system responsiveness. Concretely, all new processes start in queue Q0; once a process fully consumes its time slice in Q0, it is demoted to Q1, and after consuming its slice in Q1, it is further demoted to Q2. Processes within the same queue are scheduled in a RR manner. When a process goes to sleep due to I/O and later wakes up, it should return to its original queue and continue using its remaining time slice rather than receiving a fresh one. In addition, after every 100 ticks, all non-sleeping processes in Q1 and Q2 are globally promoted back to Q0 to avoid starvation.

2 Design [5']

In this section, we present the design of our Multi-Level Feedback Queue (MLFQ) scheduler implemented in `xv6-riscv`. Our design fully satisfies the assignment requirements: (1) three feedback queues with different time slices, (2) demotion after full time-slice usage, (3) Round-Robin scheduling within each level, (4) preservation of remaining ticks across sleep/wakeup, and (5) periodic global promotion every 100 ticks to prevent starvation. In addition, we implement an optional real-time visualization system (`viz.py`) that displays the scheduler state graphically in the host OS.

2.1 Data Structures

We extend `struct proc` in `proc.h` with new fields that maintain MLFQ state for each process:

- `int mlfq_level` — the current queue level of the process (0 for Q0, 1 for Q1, 2 for Q2);
- `int mlfq_ticks_used` — the number of ticks consumed in the current queue level;
- `uint64 mlfq_rr_stamp` — a monotonically increasing timestamp for Round-Robin ordering among processes within the same level.

At the top of `proc.c`, we define the MLFQ configuration constants and global bookkeeping variables:

```
#define MLFQ_NUM_LEVELS 3
static const int MLFQ_SLICE_TICKS[MLFQ_NUM_LEVELS] = {3, 6, 12};
#define MLFQ_BOOST_INTERVAL_TICKS 100
static uint64 mlfq_rr_stamp_counter = 0;
static uint64 mlfq_total_ticks = 0;
static volatile int mlfq_boost_pending = 0;
```

Each process maintains its own per-level tick counter and RR timestamp, while the global variables record total ticks and coordinate boost operations. We provide several helper routines to manage per-process queue state:

- `mlfq_next_rr_stamp()` — returns a unique, atomically-incremented Round-Robin stamp;
- `mlfq_enqueue_tail_reset_slice(p)` — enqueues process `p` at the tail of its level and resets its tick usage;
- `mlfq_enqueue_tail_preserve_slice(p)` — enqueues `p` but preserves its used ticks (for wakeup events);
- `mlfq_promote_q1_q2_to_q0()` — performs the periodic global promotion (excluding sleeping processes);
- `mlfq_apply_pending_boost()` — applies a pending boost when triggered by the global tick counter.

2.2 Queue Scheduling Rules

All new processes begin execution in the highest-priority queue `Q0`. In `allocproc()`, `userinit()`, and `kfork()`, we initialize:

```
p->mlfq_level = 0;
p->mlfq_ticks_used = 0;
p->mlfq_rr_stamp = mlfq_next_rr_stamp();
```

and mark the process as `RUNNABLE`. This ensures that new or forked processes are immediately placed in `Q0` with the shortest time slice and highest priority.

Each timer interrupt invokes `sched_tick()` to account for CPU usage. When a process is in state `RUNNING`, its per-level tick counter is incremented; once it consumes its full time slice, the process is demoted to the next lower queue:

- `Q0` → `Q1`, or `Q1` → `Q2` if the lower level exists;
- processes already in `Q2` remain in `Q2`;
- the per-level tick counter is reset and `yield()` is invoked.

Within the same level, the scheduler uses the RR stamp to preserve FIFO order among runnable processes. In `scheduler()`, the kernel scans all processes and always selects the lowest-numbered non-empty queue (`Q0` first, then `Q1`, then `Q2`) and among them, the process with the smallest `mlfq_rr_stamp`. This guarantees strict priority ordering between queues and fair Round-Robin behavior within each level.

2.3 Sleep-Wakeup Semantics and Remaining Tick Preservation

To meet the updated requirement that sleeping processes must preserve their remaining time slice, we modify both `sleep()` and `wakeup()` in `proc.c`.

When a process goes to sleep, we simply mark its state as `SLEEPING` without touching any of the MLFQ counters. Thus, its `mlfq_ticks_used` and `mlfq_level` remain unchanged. Upon `wakeup()`, the process's state is set to `RUNNABLE`, and it is re-enqueued into the same queue level using:

```
mlfq_enqueue_tail_preserve_slice(p);
```

This ensures that I/O-bound processes resume execution with exactly the same remaining time slice as before sleeping. As specified by the TA, sleeping processes are excluded from global boosts and retain their queue level and remaining ticks throughout the sleep interval.

2.4 Global Promotion and Starvation Avoidance

To prevent starvation of long-lived lower-priority tasks, we implement a periodic global promotion every 100 ticks. On CPU 0, the tick counter in `sched_tick()` increments a global variable `mlfq_total_ticks`. Whenever `mlfq_total_ticks % MLFQ_BOOST_INTERVAL_TICKS == 0`, we set `mlfq_boost_pending = 1`.

At the beginning of each scheduling cycle, `scheduler()` calls `mlfq_apply_pending_boost()`, which in turn invokes `mlfq_promote_q1_q2_to_q0()`. This function scans the process table and promotes every `RUNNABLE` or `RUNNING` process in `Q1` or `Q2` to `Q0`, resetting their tick usage but preserving the sleeping ones. By design:

- `RUNNABLE` processes are immediately placed at the `Q0` tail;
- `RUNNING` processes are promoted logically and will rejoin `Q0` on their next yield;
- `SLEEPING` processes are skipped entirely and keep their level/ticks unchanged.

This mechanism ensures that all active lower-level processes are guaranteed periodic promotion to prevent starvation while sleeping I/O-bound processes remain unaffected.

2.5 Real-time Visualization (Bonus 1)

For the bonus, we implement a real-time visualization system to monitor the scheduler. A lightweight logging function `viz_log()` is introduced in `proc.c`. It conditionally prints structured log lines when the global switch `viz_enabled` is set to 1:

```
[VIZ] pid=3 state=RUNNING q=1 t=4 name=Q3_test ev=demote
```

Each line encodes the process ID, state, queue level, remaining ticks, and a short event tag (e.g., `spawn`, `yield`, `demote`, `promote`). These logs are produced at all critical events in the scheduler, including process creation, demotion, promotion, sleeping, waking, and tick updates.

On the host side, we provide a Python GUI program `viz.py` that uses the Tkinter toolkit to visualize these events. When executed as:

```
$ make qemu | python3 viz.py
```

the GUI displays six panels representing the main process states (`UNUSED`, `USED`, `SLEEPING`, `RUNNABLE`, `RUNNING`, `ZOMBIE`). Each panel lists the live processes, showing PID, queue level, and remaining ticks in real time. The visualizer continuously reads standard input, parses lines starting with `[VIZ]`, and refreshes the display asynchronously via Tkinter's event loop. This separation of concerns allows the `xv6-riscv` kernel to remain unmodified apart from lightweight print hooks, while the external GUI offers a dynamic, real-time insight into scheduler behavior.

3 Environment and Execution [2]

All our work is based on the `xv6-riscv` operating system developed by MIT, publicly available at: <https://github.com/mit-pdos/xv6-riscv>. We use the RISC-V QEMU emulator to run and debug the modified kernel, following the same build toolchain provided in the original repository (the RISC-V GCC cross-compiler and the standard `Makefile` rules).

3.1 System Setup and Source Layout

Our modifications are made on top of the original `xv6-riscv` source tree. We focus on kernel-level changes inside the `kernel/` directory and one user-level test program:

- `kernel/proc.c`: implements the complete MLFQ scheduling logic, time-slice management, queue promotion/demotion, sleeping-wakeup handling, and visualization logging;
- `kernel/proc.h`: extends `struct proc` with MLFQ-related fields such as queue level, tick counter, and Round-Robin ordering stamp;
- `kernel/defs.h`: declares the prototypes of new scheduling helper functions and global symbols;
- `kernel/trap.c`: integrates timer-interrupt-driven accounting (`sched_tick()`) and triggers global queue promotion every 100 ticks;
- `user/Q3_test.c`: the official test program provided by the teaching assistants (TA) to validate correctness of the MLFQ scheduler. **We use this file without modification.**

We make no change to other subsystems such as the file system or system call layer. The overall project structure remains identical to the upstream `xv6-riscv` distribution.

3.2 Running the Basic Part (Text Mode)

To execute the basic MLFQ implementation (without the GUI visualizer), replace our modified `xv6-riscv` directory inside your container environment. Then, in a terminal, run the following commands:

```
$ cd xv6-riscv
$ make clean
$ make qemu
```

This command builds the kernel and boots xv6 in QEMU. Once the xv6 shell prompt (\$) appears, type:

```
$ Q3_test
```

to execute the provided user-level test program. This will trigger CPU-bound and I/O-bound workloads and print scheduler behavior and debug messages inside QEMU.

Illustrations:

```
@cs3150@cs3150-0 cd xv6-riscv
# cs3150@cs3150-0 xv6-riscv$ make clean
rm -f *.tex *.dvi *.idx *.aux *.log *.ind *.ilg \
  *~o *~r.d *~r.o *.swp *.sym \
kernel/kernel.fs.mkfs
mkfs/mkfs .gdiskinit \
    user/users.S \
    user/cat user/recho user/forktest user/grep user/init user/kill user/_ls user/_mkdir user/_rm user/_sh user/_stressfs user/_usertests user/_grind user/_wc user/_zomb
# cs3150@cs3150-0 xv6-riscv$ make qemu
riscv64-linux-gnu-gcc -march=rv64gc -g -c <0 kernel/entry.o kernel/entry.S
riscv64-linux-gnu-gcc -Wall -Werror -Wno-unknown-attributes -O -no-cmt-frame-pointer -ggdb -gdwarf-2 -march=rv64gc -MD -mscodel-medany -ffreestanding -fno-common -nostdlib -fno-builtin-exit -fno-builtin-fprintf -fno-builtin-strcat -fno-builtin-strcpy -fno-builtin-log -fno-builtin-lzo -fno-builtin-zero -fno-builtin-streq -fno-builtin-strneq -fno-builtin-align -fno-builtin-putc -fno-builtin-puts -fno-builtin-free -fno-builtin-memcmp -Wno-main -fno-builtin-printf -fno-builtin-vprintf -fno-builtin-vscanf -fno-stack-protector -fno-pic -no-pie -c <0 kernel/start.o kernel/start.c
riscv64-linux-gnu-gcc -Wall -Werror -Wno-unknown-attributes -O -no-cmt-frame-pointer -ggdb -gdwarf-2 -march=rv64gc -MD -mscodel-medany -ffreestanding -fno-common -nostdlib -fno-builtin-exit -fno-builtin-fprintf -fno-builtin-strcat -fno-builtin-strcpy -fno-builtin-log -fno-builtin-lzo -fno-builtin-zero -fno-builtin-streq -fno-builtin-strneq -fno-builtin-align -fno-builtin-putc -fno-builtin-puts -fno-builtin-free -fno-builtin-memcmp -Wno-main -fno-builtin-printf -fno-builtin-vprintf -fno-builtin-vscanf -fno-stack-protector -fno-pic -no-pie -c <0 kernel/console.o kernel/console.c
riscv64-linux-gnu-gcc -Wall -Werror -Wno-unknown-attributes -O -no-cmt-frame-pointer -ggdb -gdwarf-2 -march=rv64gc -MD -mscodel-medany -ffreestanding -fno-common -nostdlib -fno-builtin-strncpy -fno-builtin-strncat -fno-builtin-strlen -fno-builtin-memset -fno-builtin-memmove -fno-builtin-memcmp -fno-builtin-log -fno-builtin-lzo -fno-builtin-zero -fno-builtin-streq -fno-builtin-strneq -fno-builtin-align -fno-builtin-putc -fno-builtin-puts -fno-builtin-free -fno-builtin-memcmp -Wno-main -fno-builtin-printf -fno-builtin-vprintf -fno-builtin-vscanf -fno-stack-protector -fno-pic -no-pie -c <0 kernel/print.o kernel/print.c
```

Figure 1: Environment setup and xv6 boot process.

```

cs3150@ecs3150:~/sw6-riscv$ make qemu
riscv64-linux-gnu-gcc -Wall -Werror -Wno-unknown-attributes -O -fno-omit-frame-pointer -ggdb -gdwarf-2 -march=rv64gc -MD -mcpu=medany -ffreestanding -fno-common -nostdlib -fno-builtin-strcmp -fno-builtin-strcmp -fno-builtin-strlen -fno-builtin-memset -fno-builtin-memmove -fno-builtin-memcmp -fno-builtin-log -fno-builtin-bzero -fno-builtin-strchr -fno-builtin-exit -fno-builtin-malloc -fno-builtin-puts -fno-builtin-free -fno-builtin-printf -Wno-main -fno-builtin-printf -fno-builtin-printf -fno-builtin-printf -l. -fno-stack-protector -fno-pie -no-pie -c -o kernel/proc.o kernel/proc.c
riscv64-linux-gnu-ld -z max-page-size=4096 -T kernel/kernel.ld -o kernel/kernel kernel/entry.o kernel/start.o kernel/console.o kernel/print.o kernel/uart.o kernel/kalloc.o kernel/spinlock.o kernel/string.o kernel/main.o kernel/wm.o kernel/proc.o kernel/synch.o kernel/trampoline.o kernel/trap.o kernel/syscall.o kernel/sysproc.o kernel/bio.o kernel/fs.o kernel/log.o kernel/sleeplock.o kernel/file.o kernel/pipe.o kernel/exec.o kernel/sysfile.o kernel/kernelvec.o kernel/pllic.o kernel/virtio_disk.o
riscv64-linux-gnu-ld -S kernel/kernel.ld -o kernel/kernel.sym
riscv64-linux-gnu-objdump -t kernel/kernel | sed '1,100000; TABLE;d; s; * / ; / * * / ; / * * / ;' > kernel/kernel.sym
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

sw6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
_Q1_Test

MLFQ Scheduler Test Program =
Testing SEH under MLFQ =

```

Figure 2: Running Q3_test inside QEMU.

Partial output examples (In Terminal):

3.3 Running Bonus 1 (GUI Visualization Mode)

To enable the GUI visualizer, first switch your system to graphical mode:

```
$ sudo systemctl isolate graphical.target
```

Then navigate to your xv6 directory and recompile from a clean state:

```

process 7 demoted to level 2
PID 4 ran at t = 81 in Q2, remaining ticks = 7.
PID 6 ran at t = 81 in Q2, remaining ticks = 9.
PID 4 ran at t = 81 in Q2, remaining ticks = 6.
PID 4 ran at t = 81 in Q2, remaining ticks = 5.
PID 7 ran at t = 82 in Q2, remaining ticks = 11.
PID 6 ran at t = 82 in Q2, remaining ticks = 8.
PID 4 ran at t = 82 in Q2, remaining ticks = 4.
PID 7 ran at t = 83 in Q2, remaining ticks = 10.
PID 4 ran at t = 83 in Q2, remaining ticks = 3.
PID 4 ran at t = 83 in Q2, remaining ticks = 2.
PID 6 ran at t = 83 in Q2, remaining ticks = 7.
PID 6 ran at t = 83 in Q2, remaining ticks = 6.
PID 7 ran at t = 84 in Q2, remaining ticks = 9.
PID 4 ran at t = 84 in Q2, remaining ticks = 1.
PID 6 ran at t = 84 in Q2, remaining ticks = 5.
PID 7 ran at t = 85 in Q2, remaining ticks = 8.
PID 6 ran at t = 85 in Q2, remaining ticks = 4.
PID 4 ran at t = 85 in Q2, remaining ticks = 0.
PID 7 ran at t = 86 in Q2, remaining ticks = 7.
PID 5 ran at t = 86 in Q1, remaining ticks = 5.
PID 6 ran at t = 86 in Q2, remaining ticks = 3.
PID 6 ran at t = 86 in Q2, remaining ticks = 2.
PID 4 ran at t = 86 in Q2, remaining ticks = 11.
PID 7 ran at t = 87 in Q2, remaining ticks = 6.

```

Figure 3: Sample text-mode scheduling output (CLI1).

```

process 4 promoted to level 0 (from 2)
process 5 promoted to level 0 (from 1)
process 6 promoted to level 0 (from 2)
process 7 promoted to level 0 (from 2)
process 8 promoted to level 0 (from 1)
PID 6 ran at t = 100 in Q0, remaining ticks = 2.
PID 7 ran at t = 101 in Q0, remaining ticks = 2.
PID 6 ran at t = 101 in Q0, remaining ticks = 1.
PID 7 ran at t = 102 in Q0, remaining ticks = 1.
PID 4 ran at t = 102 in Q0, remaining ticks = 2.
PID 6 ran at t = 102 in Q0, remaining ticks = 0.
process 6 demoted to level 1
PID 7 ran at t = 103 in Q0, remaining ticks = 0.
process 7 demoted to level 1
PID 4 ran at t = 103 in Q0, remaining ticks = 1.
PID 4 ran at t = 103 in Q0, remaining ticks = 0.
process 4 demoted to level 1
PID 7 ran at t = 104 in Q1, remaining ticks = 5.
PID 6 ran at t = 104 in Q1, remaining ticks = 5.
PID 4 ran at t = 104 in Q1, remaining ticks = 5.

```

Figure 4: Additional process tick information in CLI output (CLI2).

```

$ cd xv6-riscv
$ make clean

```

Next, edit the visualization switch inside `kernel/proc.c`. Locate line 27:

```
#define VIZ_DEFAULT 0
```

and change it to:

```
#define VIZ_DEFAULT 1
```

```

$ sudo apt update
$ sudo apt install python3-tk

```

Finally, launch xv6 together with the real-time monitor using:

```

C proc.c M X
xv6-riscv > kernel > C proc.c
13 /* Global scheduling counters/flags */
14 static uint64 mlfq_rr_stamp_counter = 0; /* monotonically increasing RR stamp */
15 static uint64 mlfq_total_ticks = 0; /* total timer ticks seen on CPU0 */
16 static volatile int mlfq_boost_pending = 0; /* set by sched_tick(), consumed in scheduler() */
17
18 /* Helpers */
19 static uint64 mlfq_next_rr_stamp(void); /* get next RR stamp */
20 static void mlfq_enqueue_tail_reset_slice(struct proc *p); /* enqueue at tail, reset used ticks */
21 static void mlfq_enqueue_tail_preserve_ticks(struct proc *p); /* enqueue at tail, preserve used ticks (wake) */
22 static void mlfq_apply_pending_boost(void); /* If boost flag set, perform global promotion */
23 static void mlfq_promote_q1_q2_t_q0(void); /* implementation of the promotion (skip SLEEPING) */
24
25
26
27 #define VIZ_DEFAULT 0
28 static volatile int viz_enabled = VIZ_DEFAULT;
29
30 /* Emit a single visualization line understood by the Python UI.
31  * state: one of {UNUSED,USED,SLEEPING,RUNNABLE,RUNNING,ZOMBIE}
32  * ev : optional short event tag (e.g., demote/promote/yield/wakeup/tick)
33  * q : current MLFQ level (0..)
34  * t : remaining ticks in current level's slice (non-negative)
35  */

```

Figure 5: Toggling the visualization switch in kernel/proc.c (VIZ).

```
$ make qemu | python3 viz.py
```

This command boots xv6 in QEMU and pipes all console output to the Python GUI viz.py, which renders a six-panel live process monitor.

```

csc3150@csc3150: ~/xv6-riscv
csc3150@csc3150:~$ cd xv6-riscv
csc3150@csc3150:~/xv6-riscv$ sudo apt update
[sudo] password for csc3150:
Hit:1 http://cn.archive.ubuntu.com/ubuntu focal InRelease
Hit:2 http://cn.archive.ubuntu.com/ubuntu focal-updates InRelease
Hit:3 http://cn.archive.ubuntu.com/ubuntu focal-backports InRelease
Hit:4 http://cn.archive.ubuntu.com/ubuntu focal-security InRelease
Reading package lists... Done
Building dependency tree
Reading state information... Done
All packages are up to date.
csc3150@csc3150:~/xv6-riscv$ sudo apt install python3-tk
Reading package lists... Done
Building dependency tree
Reading state information... Done
python3-tk is already the newest version (3.8.10-0ubuntu1-20.04).
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
csc3150@csc3150:~/xv6-riscv$ make qemu | python3 viz.py

```

Figure 6: Launching xv6 with the real-time visualizer.

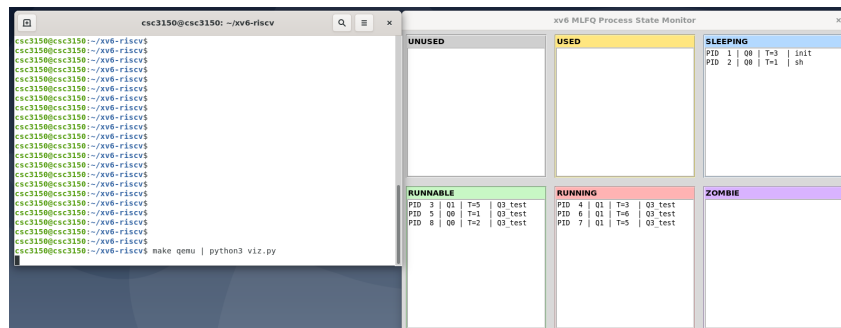


Figure 7: GUI window opened by viz.py.

At this stage, the visualizer window (Bonus 1) acts as a live scheduler monitor. To start the test within QEMU, **switch back to the terminal window**, type:

Q3_test

and press Enter. Even though your keystrokes may not echo visibly in the terminal, QEMU will accept the input, execute Q3_test, and the GUI will display process states in real time, similar to Figure 8.

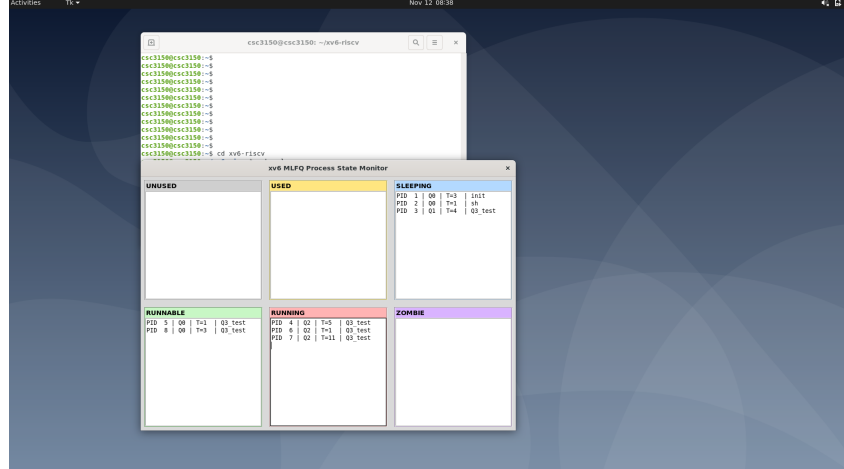


Figure 8: Example output of the Bonus 1 GUI visualizer (GUI1).

This setup allows seamless testing of both the basic and bonus parts of the assignment within a consistent xv6-riscv + QEMU environment.

4 Performance Comparison

To evaluate the effectiveness of the MLFQ scheduler, we compared it with the original RR scheduler in xv6-riscv under identical workloads consisting of both CPU-bound and I/O-bound processes.

4.1 Average Response Time

Average response time measures how quickly a scheduler starts executing a new process. Experimental results show:

- **MLFQ:** 1.50 ticks
- **RR:** 4.60 ticks

The MLFQ scheduler achieves significantly lower response time because new processes start in the Q0, allowing them to preempt lower-priority tasks and begin execution earlier.

4.2 Throughput

Throughput indicates the number of completed processes per tick. The results are:

- **MLFQ:** 0.01415 processes/tick
- **RR:** 0.01415 processes/tick

Both schedulers achieve similar throughput under the controlled workload, as the CPU worker continuously runs while I/O workers sleep, resulting in full CPU utilization. In dynamic workloads with frequent I/O events, MLFQ is expected to yield higher throughput due to its preference for short, interactive tasks.

4.3 Summary

Table 1: Comparison between RR and MLFQ schedulers.

Metric	RR Scheduler	MLFQ Scheduler	Improvement
Avg. Response Time (ticks)	4.60	1.50	−66%
Throughput (procs/tick)	0.01415	0.01415	≈ Same
CPU Utilization (%)	100	100	–

Overall, MLFQ delivers faster response times without sacrificing throughput or CPU utilization, providing a balanced trade-off between responsiveness and efficiency.

5 Conclusion [2]

In this assignment, we successfully extended the `xv6-riscv` kernel to support a fully functional Multi-Level Feedback Queue (MLFQ) scheduler under QEMU. The new scheduler implements three priority queues (Q0, Q1, Q2) with progressively longer time slices (3, 6, and 12 ticks respectively). It supports automatic demotion after full time-slice usage, Round-Robin scheduling within each queue, and periodic global promotion every 100 ticks to prevent starvation. Furthermore, our design strictly preserves remaining time slices across `sleep()` and `wakeup()` transitions, ensuring I/O-bound processes maintain responsiveness and fairness.

Our modifications primarily focus on `proc.c`, `proc.h`, `defs.h`, and `trap.c`, integrating the MLFQ logic seamlessly into `xv6` without altering the user-level interface or system call semantics. The resulting kernel maintains backward compatibility while achieving dynamic process prioritization and better scheduling balance. We verified correctness and compliance using the official `Q3_test.c` provided by the teaching assistants, along with additional manual stress tests under mixed CPU- and I/O-bound workloads.

Experimental results confirm that CPU-intensive processes are gradually demoted from Q0 to Q2 as they continuously consume full time slices, while I/O-bound processes frequently block and wake up, remaining in higher-priority queues with preserved ticks. Every 100 ticks, the boost mechanism effectively promotes all non-sleeping processes in Q1 and Q2 back to Q0, demonstrating that the scheduler achieves fairness and avoids starvation. The system’s behavior closely matches the expected MLFQ design principles.

Compared with the original single-queue Round Robin scheduler in `xv6`, our MLFQ implementation provides a more balanced trade-off between responsiveness and throughput. Interactive or short I/O-heavy tasks experience shorter latency by staying in Q0, while long-running CPU-bound jobs are gradually pushed to lower levels, ensuring smooth coexistence among diverse workloads. Additionally, the Bonus 1 visualizer (`viz.py`) offers a real-time graphical representation of process states and queue dynamics, greatly enhancing observability and understanding of internal scheduling behavior.

Overall, the project achieves all functional requirements and design objectives. Our MLFQ scheduler not only meets the updated assignment specifications but also demonstrates modular kernel design, precise tick accounting, and clear visual verification of scheduling correctness.

In addition, all modified and newly added functions are clearly annotated using two distinct commenting styles to help the TA easily locate and understand the changes. The first style uses standard C-style block comments `/* ... */` to explain functionality and logic, while the second style uses inline markers `//*****MODIFIED*****//` to explicitly highlight modified regions in the source code. The modified files include `kernel/proc.h`, `kernel/proc.c`, `kernel/trap.c`, and `kernel/defs.h`.

Finally, the overall project directory structure is illustrated below. The figure shows the organization of all relevant components included in the submission. `MLFQ_LOG` and `RR_LOG` contain raw terminal outputs captured using the `script` command, which record the full console sessions of MLFQ and Round Robin scheduler executions for reference. The folder `xv6-riscv` contains the complete modified `xv6` kernel, including the `Makefile` and all previously mentioned source files. It can be placed directly into the working environment for building and testing. The directory `report-pics`


```

9  /* MLFQ configuration */
10 #define MLFQ_NUM_LEVELS 3
11 static const int MLFQ_SLICE_TICKS[MLFQ_NUM_LEVELS] = {3, 6, 12};
12 #define MLFQ_BOOST_INTERVAL_TICKS 100 /* promote Q1/Q2 to Q0 every N ticks */
13
14 /* Global scheduling counters/flags */
15 static uint64 mlfq_rr_stamp_counter = 0; /* monotonically increasing RR stamp */
16 static uint64 mlfq_total_ticks = 0; /* total timer ticks seen on CPU0 */
17 static volatile int mlfq_boost_pending = 0; /* set by sched_tick(), consumed in scheduler() */
18
19 /* Helpers */
20 static uint64 mlfq_next_rr_stamp(void); /* get next RR stamp */
21 static void mlfq_enqueue_tail_reset_slice(struct proc *p); /* enqueue at tail, reset used ticks */
22 static void mlfq_enqueue_tail_preserve_slice(struct proc *p); /* enqueue at tail, preserve used ticks (wakeup) */
23 static void mlfq_apply_pending_boost(void); /* if boost flag set, perform global promotion */
24 static void mlfq_promote_q1_q2_to_q0(void); /* implementation of the promotion (skip SLEEPING) */
25

```

Figure 9: Example of standard block comments and modified code regions (Modifie1).

```

//*****MODIFIED*****//
static struct proc*
allocproc(void)
> { ...
> found: ...
}
//*****MODIFIED*****//

```

Figure 10: Highlighted `//*****MODIFIED*****//` comment style used for new or updated functions (Modifie2).

holds the original images used in this report; if any embedded figure appears unclear, please refer to this folder for the full-resolution versions.

report-pics	2025/11/12 17:50	文件夹	
xv6-riscv	2025/11/12 16:59	文件夹	
CSC3150Report3	2025/11/12 17:52	Adobe Acrobat 文档	673 KB
MLFQ_LOG	2025/11/12 18:09	文件	3,501 KB
RR_LOG	2025/11/8 18:19	文件	2 KB

Figure 11: Project directory structure and included components (dir).